

Topological Mapping of Distributed State: A Static Analysis Framework for Detecting Race Conditions in Saga-Based Microservices

Research Division Automated Software Engineering Lab

Abstract

The transition from monolithic architectures to distributed microservices has introduced a critical verification gap: the loss of ACID transaction isolation. In Saga-based architectures, the interleaving of concurrent asynchronous events can lead to non-deterministic state anomalies, specifically distributed data races. This paper introduces the “Method of Maps,” a formal static analysis framework that models the microservice ecosystem as a graph of topological mappings between external API entry points and internal persistence layers. By constructing formal definitions for the *Endpoint Map* (Φ) and the *Data Map* (Δ), we derive an *Interference Matrix* that mathematically identifies potential race conditions without requiring runtime instrumentation. We validate this theoretical framework through a Java-based implementation applied to the Fudan Train Ticket benchmark.

I. INTRODUCTION

In monolithic systems, concurrency control is often managed via database locking (Pessimistic/Optimistic) or thread synchronization primitives. However, in microservices employing the *Database-per-Service* pattern, these mechanisms are strictly local. A business transaction that spans multiple services (a Saga) lacks global isolation (the ‘I’ in ACID).

Formally, a distributed data race occurs when two concurrent execution traces T_1, T_2 access a shared logical entity e , where $W(T_1) \cap (R(T_2) \cup W(T_2)) \neq \emptyset$, and no happened-before relation $T_1 \rightarrow T_2$ exists to order these accesses.

II. MATHEMATICAL FORMALISM: THE METHOD OF MAPS

We define a Microservice Architecture Σ as a tuple:

$$\Sigma = \langle \mathcal{S}, \mathcal{E}, \mathcal{D}, \mathcal{M} \rangle \quad (1)$$

where:

- $\mathcal{S} = \{s_1, \dots, s_n\}$ is the set of microservices.
- $\mathcal{E} = \{e_1, \dots, e_m\}$ is the set of exposed endpoints (HTTP/RPC).
- $\mathcal{D} = \{d_1, \dots, d_k\}$ is the set of persistent data entities (Tables/Collections).
- $\mathcal{M}$ is the messaging alphabet used for asynchronous communication.

A. Map 1: The Data Map (Δ)

The Data Map function Δ defines the state mutation capabilities of a specific code artifact (Repository method). Let $\mathcal{R}$ be the set of repository methods.

$$\Delta : \mathcal{R} \rightarrow \mathcal{D} \times \{\text{READ}, \text{WRITE}, \text{DELETE}\} \quad (2)$$

For a repository method $r \in \mathcal{R}$, $\Delta(r) = \langle d, \omega \rangle$ implies that executing r results in operation ω on entity d .

B. Map 2: The Endpoint Map (Φ)

The Endpoint Map Φ represents the transitive closure of the call graph rooted at an endpoint. It maps an external entry point to the set of state mutations it triggers.

$$\Phi : \mathcal{E} \rightarrow \mathcal{P}(\mathcal{D} \times \Omega) \quad (3)$$

where $\Omega = \{\text{READ}, \text{WRITE}\}$. $\Phi(e_i)$ is computed by traversing the Abstract Syntax Tree (AST) from the controller e_i through the service layer to any reachable repository methods defined in Δ .

C. The Interference Condition

We define a **Potential Distributed Data Race** between two endpoints $e_a, e_b \in \mathcal{E}$ (where $a \neq b$) if their execution creates an interference on a shared entity $d \in \mathcal{D}$.

Let $\Phi(e_a) = \mathcal{O}_a$ and $\Phi(e_b) = \mathcal{O}_b$. A race exists if:

$$\exists \langle d, \omega_a \rangle \in \mathcal{O}_a, \langle d, \omega_b \rangle \in \mathcal{O}_b : (\omega_a = \text{WRITE} \vee \omega_b = \text{WRITE}) \quad (4)$$

This predicate identifies purely static candidates. To verify the race, we must also prove the absence of synchronization:

$$\text{Race}(e_a, e_b) \iff \text{Interference}(e_a, e_b) \wedge \neg \text{Locked}(d) \quad (5)$$

III. ALGORITHMIC IMPLEMENTATION

To compute Φ and Δ statically, we utilize an Abstract Syntax Tree (AST) traversal algorithm.

Algorithm 1 Method of Maps: Race Extraction

```

1:  $Map_{Data} \leftarrow \emptyset$ 
2:  $Map_{Endpoint} \leftarrow \emptyset$ 
3:  $Candidates \leftarrow \emptyset$ 
   {Phase 1: Build Data Map}
4: for all  $repo \in \mathcal{S}.Repositories$  do
5:    $entity \leftarrow ResolveGenericType(repo)$ 
6:    $Map_{Data}[repo] \leftarrow entity$ 
7: end for
   {Phase 2: Build Endpoint Map}
8: for all  $endpoint \in \mathcal{S}.Endpoints$  do
9:    $callGraph \leftarrow BuildCallGraph(endpoint)$ 
10:   $mutations \leftarrow \emptyset$ 
11:  for all  $node \in callGraph$  do
12:    if  $node \in Map_{Data}$  then
13:       $mutations.add(Map_{Data}[node])$ 
14:    end if
15:  end for
16:   $Map_{Endpoint}[endpoint] \leftarrow mutations$ 
17: end for
   {Phase 3: Detect Interference}
18: for all  $e_i, e_j \in Map_{Endpoint}$  do
19:    $intersection \leftarrow Map_{Endpoint}[e_i] \cap Map_{Endpoint}[e_j]$ 
20:   if  $intersection \neq \emptyset \wedge HasWrite(intersection)$  then
21:      $Candidates.add(\langle e_i, e_j, intersection \rangle)$ 
22:   end if
23: end for
24: return  $Candidates$ 

```

IV. CONCLUSION

The Method of Maps provides a deterministic approach to reducing the search space for distributed concurrency bugs. By formalizing the relationship between endpoints and persistence, we can identify the “Race Surface” of a microservice architecture before deployment.